

Lab Context and Topic Summary

Start here: what the lab is, what each section contains, and the mental model for reading the reference.

Purpose of this field reference

This book documents a Containerlab-based **VXLAN/EVPN 5-stage Clos lab** with 2 super-spines, 4 spines, 8 leaves, 2 border-leaves, 2 simulated ISPs, 1 FRR internet router, 3 tenant VRFs, and 32 host PCs. Use the first pages to understand the topology and traffic flow, then use the reference sections for exact IPs, ASNs, VNIs, verification commands, troubleshooting, and config templates.

Core mental model

Underlay

IPv4 eBGP gives physical-link and Loopback0 reachability. If VTEP routes are missing, VXLAN traffic fails.

Overlay

EVPN advertises host MAC/IP, VTEP membership, and internet prefixes. Spines/super-spines act as route reflectors.

Data plane

Leaves encapsulate traffic into VXLAN using local and remote VTEP Loopback0 addresses. Transit tiers route the outer IP packet.

Topic map for the original reference book

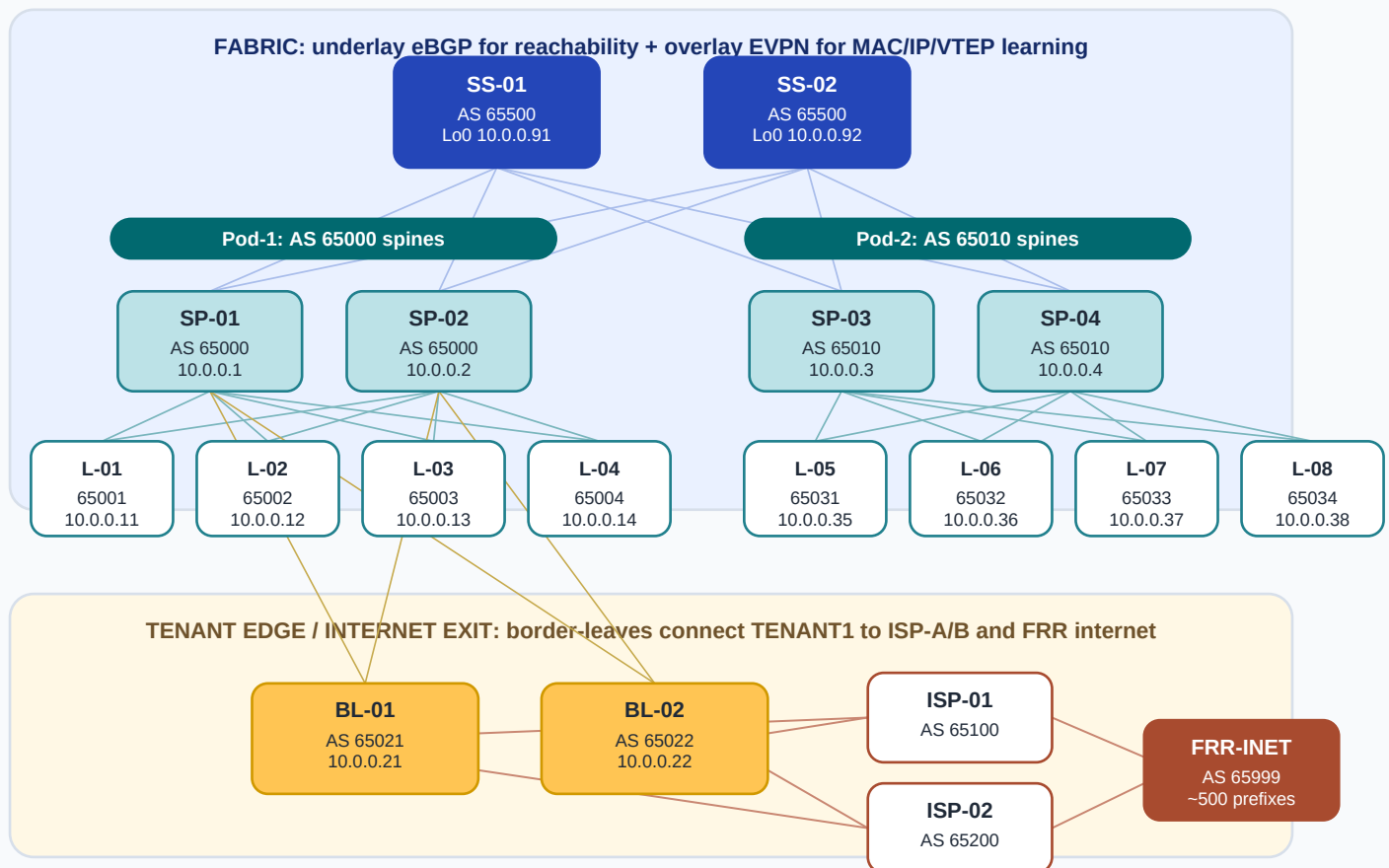
Part	What it covers	Why it matters
1	At-a-glance lab facts	Fast inventory: containers, roles, tenants, VNIs, BGP settings.
2	Device inventory and access	Hostnames, ASNs, loopbacks, management IPs, SSH access.
3	IP address tables	Exact underlay P2P, ISP, and FRR link addressing.
4	BGP session map	Underlay eBGP and EVPN overlay peering by tier.
5	VLAN, VNI, VRF reference	Tenant/VLAN/L2 VNI/L3 VNI/gateway and route-target mapping.
6	Host PC reference	pc1-pc32 placement, IPs, gateways, and ping matrix.
7	EVPN route types	Type-2, Type-3, Type-5 meaning in this lab.
8-9	Show and batch commands	Role-based and all-node verification commands.
10-11	Troubleshooting and silent rejection	Stepwise fault isolation and cEOS gotchas.
12-14	Operations, walk-through, templates	Containerlab operations, packet path, and config templates.

Suggested reading path

1. Read the quick-start topology and traffic-flow pages. 2. Use Parts 1-6 for exact lab inventory and addressing. 3. Use Parts 7-11 while verifying or troubleshooting. 4. Use Parts 12-14 when operating, rebuilding, or templating the lab.

5-Stage Clos Topology Map

Physical/logical fabric view: underlay reachability below, EVPN/VXLAN services above.



Host placement mental model

Pod-1: pc1-pc16 sit behind leaf-01 to leaf-04. Each leaf carries TENANT1 VLAN10/20, TENANT2 VLAN30, and TENANT3 VLAN50.

What the tiers do

Super-spines: inter-pod EVPN RR.
 Spines: pod EVPN RR + ECMP next-hop.
 Leaves/BLs: VTEPs, anycast gateway, VXLAN encap/decap.

Important loopback idea

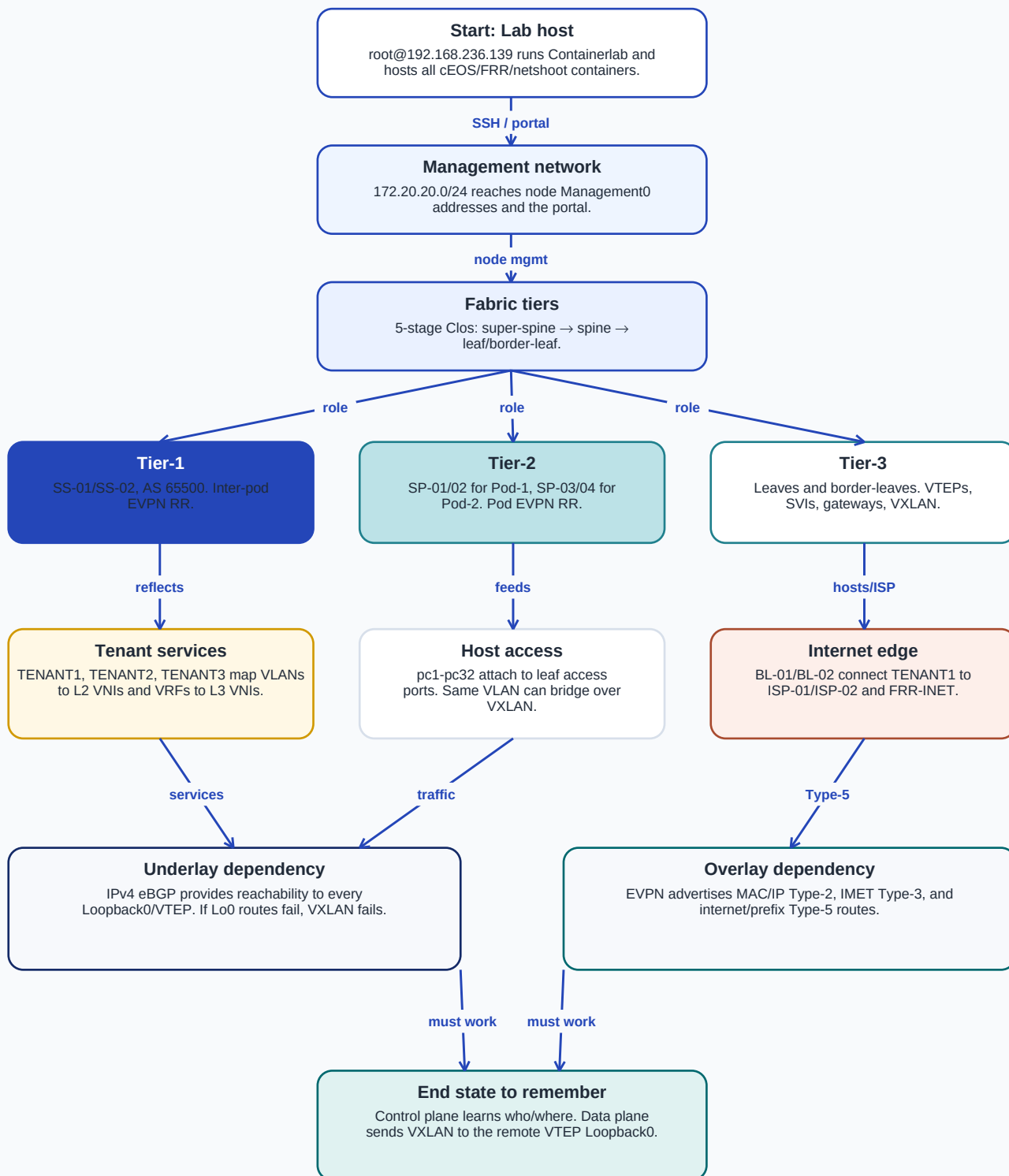
Every leaf and border-leaf uses Loopback0 as the VTEP source. EVPN advertises MAC/IP, IMET, and prefix routes with VTEP loopbacks as next-hops, so underlay reachability to Lo0s must be stable first.

Internet edge idea

Only TENANT1 has internet propagation in this lab. Border-leaves learn ISP routes in VRF TENANT1 and redistribute selected prefixes/defaults into EVPN Type-5.

Topology Understanding Flowchart

Use this when you want to explain the lab from management access to fabric tiers and tenant services.

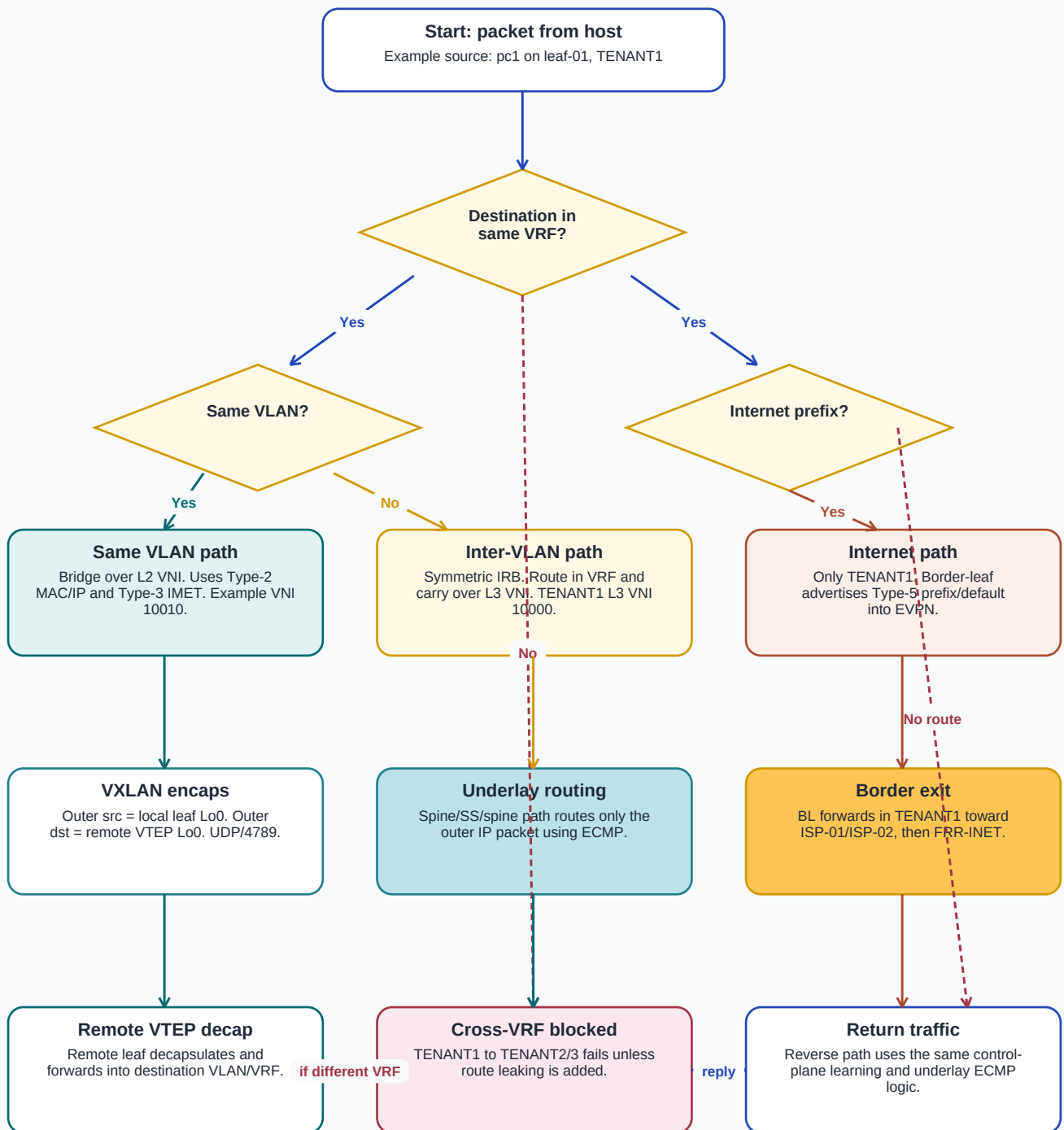


How to explain the topology in one minute

Start with the lab host and management subnet, then identify the three fabric tiers. After that, explain that leaves are VTEPs, border-leaves are internet exits for TENANT1, and EVPN tells VTEPs where remote hosts/prefixes live.

Traffic Flow Decision Flowchart

Start with the source and destination, then follow the decision path to know which VNI, route type, and devices are involved.



Quick troubleshooting mapping

If the same-VLAN path fails, check Type-2/Type-3 and VXLAN VTEP tables. If inter-VLAN fails, check SVI/VRF/L3 VNI. If internet fails, check border-leaf ISP BGP, Type-5 routes, and TENANT1 VRF routing.

VXLAN / EVPN Lab — Field Reference Book

Lab host: `root@192.168.236.139`

Topology: 5-stage Clos · 2 super-spines · 4 spines · 8 leaves · 2 border-leaves · 2 ISPs · 1 FRR internet · 32 host PCs

Software: cEOS-lab 4.36.0.1F · Containerlab · Ubuntu 26.04

Part 1 — At a Glance

Metric	Value
Total containers	51
cEOS routers	19 (2 SS + 4 SP + 8 L + 2 BL + 2 ISP + 1 FRR)
Host PCs	32 (Linux netshoot)
Pods	2
VRFs / tenants	3 (TENANT1, TENANT2, TENANT3)
Internet prefixes	~500 (simulated DFZ via frr-inet)
BGP AS range	65000 – 65999
VTEP source	Loopback0 of every leaf and border-leaf
Anycast gateway MAC	<code>00:1c:73:00:00:99</code> (all VTEPs)
VXLAN UDP port	4789
BGP ECMP max-paths	4
EVPN multihop TTL	3
Management subnet	172.20.20.0/24
Portal URL	<code>http://192.168.236.139:5000</code>

Part 2 — Device Inventory

Network Nodes

Container Name	Hostname	ASN	Loopback0	Mgmt IP	Role
clab-vxlan-evpn-super-spine-01	SUPER-SPINE-01	65500	10.0.0.91/32	172.20.20.91	Tier-1 · Inter-pod EVPN RR
clab-vxlan-evpn-super-spine-02	SUPER-SPINE-02	65500	10.0.0.92/32	172.20.20.92	Tier-1 · Inter-pod EVPN RR
clab-vxlan-evpn-spine-01	SPINE1	65000	10.0.0.1/32	172.20.20.101	Tier-2 Pod-1 · EVPN RR
clab-vxlan-evpn-spine-02	SPINE2	65000	10.0.0.2/32	172.20.20.102	Tier-2 Pod-1 · EVPN RR
clab-vxlan-evpn-spine-03	SPINE3	65010	10.0.0.3/32	172.20.20.103	Tier-2 Pod-2 · EVPN RR
clab-vxlan-evpn-spine-04	SPINE4	65010	10.0.0.4/32	172.20.20.104	Tier-2 Pod-2 · EVPN RR
clab-vxlan-evpn-leaf-01	LEAF1	65001	10.0.0.11/32	172.20.20.111	Pod-1 VTEP
clab-vxlan-evpn-leaf-02	LEAF2	65002	10.0.0.12/32	172.20.20.112	Pod-1 VTEP
clab-vxlan-evpn-leaf-03	LEAF3	65003	10.0.0.13/32	172.20.20.113	Pod-1 VTEP
clab-vxlan-evpn-leaf-04	LEAF4	65004	10.0.0.14/32	172.20.20.114	Pod-1 VTEP
clab-vxlan-evpn-leaf-05	LEAF5	65031	10.0.0.35/32	172.20.20.131	Pod-2 VTEP
clab-vxlan-evpn-leaf-06	LEAF6	65032	10.0.0.36/32	172.20.20.132	Pod-2 VTEP
clab-vxlan-evpn-leaf-07	LEAF7	65033	10.0.0.37/32	172.20.20.133	Pod-2 VTEP
clab-vxlan-evpn-leaf-08	LEAF8	65034	10.0.0.38/32	172.20.20.134	Pod-2 VTEP
clab-vxlan-evpn-border-leaf-01	BORDER-LEAF-01	65021	10.0.0.21/32	172.20.20.121	Pod-1 Border VTEP
clab-vxlan-evpn-border-leaf-02	BORDER-LEAF-02	65022	10.0.0.22/32	172.20.20.122	Pod-1 Border VTEP
clab-vxlan-evpn-isp-01	isp-01	65100	—	172.20.20.250	Simulated ISP-A
clab-vxlan-evpn-isp-02	isp-02	65200	—	172.20.20.251	Simulated ISP-B
clab-vxlan-evpn-frr-inet	frr-inet	65999	—	172.20.20.252	FRR · ~500 internet prefixes

SSH Quick Access

```
# Add to ~/.ssh/config on your workstation:
Host 172.20.20.*
  ProxyJump root@192.168.236.139
  User admin
  StrictHostKeyChecking no
  UserKnownHostsFile /dev/null
```

Node	IP	Credentials
super-spine-01	172.20.20.91	admin / admin
super-spine-02	172.20.20.92	admin / admin
spine-01	172.20.20.101	admin / admin
spine-02	172.20.20.102	admin / admin
spine-03	172.20.20.103	admin / admin
spine-04	172.20.20.104	admin / admin
leaf-01	172.20.20.111	admin / admin
leaf-02	172.20.20.112	admin / admin
leaf-03	172.20.20.113	admin / admin
leaf-04	172.20.20.114	admin / admin
leaf-05	172.20.20.131	admin / admin
leaf-06	172.20.20.132	admin / admin
leaf-07	172.20.20.133	admin / admin
leaf-08	172.20.20.134	admin / admin
border-leaf-01	172.20.20.121	admin / admin
border-leaf-02	172.20.20.122	admin / admin
isp-01	172.20.20.250	admin / admin
isp-02	172.20.20.251	admin / admin
frr-inet	172.20.20.252	<code>docker exec -it clab-vxlan-evpn-frr-inet vtysh</code>

Part 3 — IP Address Tables

3.1 Underlay P2P Links — Tier-1 ↔ Tier-2 (Super-Spine ↔ Spine)

Link	Subnet	Super-Spine IP	Spine IP	Interfaces
SS-01 ↔ SP-01	10.2.1.0/31	10.2.1.0	10.2.1.1	SS01:eth1 ↔ SP01:eth7
SS-01 ↔ SP-02	10.2.1.2/31	10.2.1.2	10.2.1.3	SS01:eth2 ↔ SP02:eth7
SS-02 ↔ SP-01	10.2.2.0/31	10.2.2.0	10.2.2.1	SS02:eth1 ↔ SP01:eth8
SS-02 ↔ SP-02	10.2.2.2/31	10.2.2.2	10.2.2.3	SS02:eth2 ↔ SP02:eth8
SS-01 ↔ SP-03	10.2.3.0/31	10.2.3.0	10.2.3.1	SS01:eth3 ↔ SP03:eth7
SS-01 ↔ SP-04	10.2.3.2/31	10.2.3.2	10.2.3.3	SS01:eth4 ↔ SP04:eth7
SS-02 ↔ SP-03	10.2.4.0/31	10.2.4.0	10.2.4.1	SS02:eth3 ↔ SP03:eth8
SS-02 ↔ SP-04	10.2.4.2/31	10.2.4.2	10.2.4.3	SS02:eth4 ↔ SP04:eth8

3.2 Underlay P2P Links — Tier-2 ↔ Tier-3 Pod-1 (Spine ↔ Leaf)

Link	Subnet	Spine IP	Leaf IP	Interfaces
SP-01 ↔ L-01	10.1.1.0/31	10.1.1.0	10.1.1.1	SP01:eth1 ↔ L01:eth1
SP-01 ↔ L-02	10.1.1.2/31	10.1.1.2	10.1.1.3	SP01:eth2 ↔ L02:eth1
SP-01 ↔ L-03	10.1.1.4/31	10.1.1.4	10.1.1.5	SP01:eth3 ↔ L03:eth1
SP-01 ↔ L-04	10.1.1.6/31	10.1.1.6	10.1.1.7	SP01:eth4 ↔ L04:eth1
SP-01 ↔ BL-01	10.1.1.8/31	10.1.1.8	10.1.1.9	SP01:eth5 ↔ BL01:eth1
SP-01 ↔ BL-02	10.1.1.10/31	10.1.1.10	10.1.1.11	SP01:eth6 ↔ BL02:eth1
SP-02 ↔ L-01	10.1.2.0/31	10.1.2.0	10.1.2.1	SP02:eth1 ↔ L01:eth2
SP-02 ↔ L-02	10.1.2.2/31	10.1.2.2	10.1.2.3	SP02:eth2 ↔ L02:eth2
SP-02 ↔ L-03	10.1.2.4/31	10.1.2.4	10.1.2.5	SP02:eth3 ↔ L03:eth2
SP-02 ↔ L-04	10.1.2.6/31	10.1.2.6	10.1.2.7	SP02:eth4 ↔ L04:eth2
SP-02 ↔ BL-01	10.1.2.8/31	10.1.2.8	10.1.2.9	SP02:eth5 ↔ BL01:eth2
SP-02 ↔ BL-02	10.1.2.10/31	10.1.2.10	10.1.2.11	SP02:eth6 ↔ BL02:eth2

3.3 Underlay P2P Links — Tier-2 ↔ Tier-3 Pod-2 (Spine ↔ Leaf)

Link	Subnet	Spine IP	Leaf IP	Interfaces
SP-03 ↔ L-05	10.1.3.0/31	10.1.3.0	10.1.3.1	SP03:eth1 ↔ L05:eth1
SP-03 ↔ L-06	10.1.3.2/31	10.1.3.2	10.1.3.3	SP03:eth2 ↔ L06:eth1
SP-03 ↔ L-07	10.1.3.4/31	10.1.3.4	10.1.3.5	SP03:eth3 ↔ L07:eth1
SP-03 ↔ L-08	10.1.3.6/31	10.1.3.6	10.1.3.7	SP03:eth4 ↔ L08:eth1
SP-04 ↔ L-05	10.1.4.0/31	10.1.4.0	10.1.4.1	SP04:eth1 ↔ L05:eth2
SP-04 ↔ L-06	10.1.4.2/31	10.1.4.2	10.1.4.3	SP04:eth2 ↔ L06:eth2
SP-04 ↔ L-07	10.1.4.4/31	10.1.4.4	10.1.4.5	SP04:eth3 ↔ L07:eth2
SP-04 ↔ L-08	10.1.4.6/31	10.1.4.6	10.1.4.7	SP04:eth4 ↔ L08:eth2

3.4 ISP / Internet Links

Link	Subnet	BL / ISP IP	ISP / FRR IP	Interfaces
BL-01 ↔ ISP-01	100.64.0.0/30	BL01: 100.64.0.2	ISP01: 100.64.0.1	BL01:eth3 ↔ ISP01:eth1
BL-02 ↔ ISP-01	100.64.0.4/30	BL02: 100.64.0.6	ISP01: 100.64.0.5	BL02:eth3 ↔ ISP01:eth2
BL-01 ↔ ISP-02	100.64.1.0/30	BL01: 100.64.1.2	ISP02: 100.64.1.1	BL01:eth4 ↔ ISP02:eth1
BL-02 ↔ ISP-02	100.64.1.4/30	BL02: 100.64.1.6	ISP02: 100.64.1.5	BL02:eth4 ↔ ISP02:eth2
ISP-01 ↔ frr-inet	10.99.0.0/30	ISP01: 10.99.0.2	frr: 10.99.0.1	ISP01:eth5 ↔ frr-inet:eth1
ISP-02 ↔ frr-inet	10.99.0.4/30	ISP02: 10.99.0.6	frr: 10.99.0.5	ISP02:eth5 ↔ frr-inet:eth2

3.5 ISP Loopbacks (reachable from TENANT1 after internet propagation)

ISP	Loopback(s)	Notes
isp-01 (AS 65100)	1.1.1.1/32, 8.8.8.8/32	Advertised into TENANT1 BGP table
isp-02 (AS 65200)	2.2.2.2/32, 8.8.8.8/32	Advertised into TENANT1 BGP table

Part 4 — BGP Session Map

4.1 Underlay eBGP (IPv4 unicast — loopback reachability)

```

frr-inet (65999)
  eBGP ↔ isp-01 (65100)    [10.99.0.0/30]
  eBGP ↔ isp-02 (65200)    [10.99.0.4/30]

isp-01 (65100)
  eBGP ↔ border-leaf-01 TENANT1 VRF [100.64.0.0/30]
  eBGP ↔ border-leaf-02 TENANT1 VRF [100.64.0.4/30]

isp-02 (65200)
  eBGP ↔ border-leaf-01 TENANT1 VRF [100.64.1.0/30]
  eBGP ↔ border-leaf-02 TENANT1 VRF [100.64.1.4/30]

super-spine-01/02 (65500)      Pod-1 spines (65000)
  eBGP ↔ spine-01              [10.2.1.0/31]
  eBGP ↔ spine-02              [10.2.1.2/31 / 10.2.2.2/31]

super-spine-01/02 (65500)      Pod-2 spines (65010)
  eBGP ↔ spine-03              [10.2.3.0/31]
  eBGP ↔ spine-04              [10.2.3.2/31 / 10.2.4.2/31]

spine-01/02 (65000)            Pod-1 VTEPs
  eBGP ↔ leaf-01 (65001)       [10.1.1.0/31 · 10.1.2.0/31]
  eBGP ↔ leaf-02 (65002)       [10.1.1.2/31 · 10.1.2.2/31]
  eBGP ↔ leaf-03 (65003)       [10.1.1.4/31 · 10.1.2.4/31]
  eBGP ↔ leaf-04 (65004)       [10.1.1.6/31 · 10.1.2.6/31]
  eBGP ↔ border-leaf-01        [10.1.1.8/31 · 10.1.2.8/31]
  eBGP ↔ border-leaf-02        [10.1.1.10/31 · 10.1.2.10/31]

spine-03/04 (65010)            Pod-2 VTEPs
  eBGP ↔ leaf-05 (65031)       [10.1.3.0/31 · 10.1.4.0/31]
  eBGP ↔ leaf-06 (65032)       [10.1.3.2/31 · 10.1.4.2/31]
  eBGP ↔ leaf-07 (65033)       [10.1.3.4/31 · 10.1.4.4/31]
  eBGP ↔ leaf-08 (65034)       [10.1.3.6/31 · 10.1.4.6/31]

```

4.2 EVPN Overlay eBGP (AF EVPN — multihop loopback-to-loopback)

Tier-1 (super-spines) reflect for ALL pods:

```
SS-01 (10.0.0.91) ↔ SP-01 (10.0.0.1) AS 65000
SS-01           ↔ SP-02 (10.0.0.2) AS 65000
SS-01           ↔ SP-03 (10.0.0.3) AS 65010
SS-01           ↔ SP-04 (10.0.0.4) AS 65010
SS-02 mirrors all of the above
```

Tier-2 Pod-1 (spines) reflect for pod-1:

```
SP-01 (10.0.0.1) ↔ L-01 (10.0.0.11) AS 65001
SP-01           ↔ L-02 (10.0.0.12) AS 65002
SP-01           ↔ L-03 (10.0.0.13) AS 65003
SP-01           ↔ L-04 (10.0.0.14) AS 65004
SP-01           ↔ BL-01 (10.0.0.21) AS 65021
SP-01           ↔ BL-02 (10.0.0.22) AS 65022
SP-02 mirrors all of the above
```

Tier-2 Pod-2 (spines) reflect for pod-2:

```
SP-03 (10.0.0.3) ↔ L-05 (10.0.0.35) AS 65031
SP-03           ↔ L-06 (10.0.0.36) AS 65032
SP-03           ↔ L-07 (10.0.0.37) AS 65033
SP-03           ↔ L-08 (10.0.0.38) AS 65034
SP-04 mirrors all of the above
```

Key attribute on super-spines: next-hop-unchanged

- Preserves original VTEP Lo0 as BGP next-hop across pod boundary
- VXLAN data plane stays direct leaf-to-leaf (super-spines never carry VXLAN frames)

4.3 BGP AS Summary

AS	Nodes	Role
65000	spine-01, spine-02	Pod-1 underlay + EVPN RR
65001	leaf-01	Pod-1 VTEP
65002	leaf-02	Pod-1 VTEP
65003	leaf-03	Pod-1 VTEP
65004	leaf-04	Pod-1 VTEP
65010	spine-03, spine-04	Pod-2 underlay + EVPN RR
65021	border-leaf-01	Border VTEP
65022	border-leaf-02	Border VTEP
65031	leaf-05	Pod-2 VTEP
65032	leaf-06	Pod-2 VTEP
65033	leaf-07	Pod-2 VTEP
65034	leaf-08	Pod-2 VTEP
65100	isp-01	Simulated ISP-A
65200	isp-02	Simulated ISP-B
65500	super-spine-01, super-spine-02	Inter-pod EVPN RR
65999	frr-inet	Simulated internet (FRR)

Part 5 — VLAN / VNI / VRF Reference

5.1 Complete Mapping Table

VRF	VLAN	VNI (L2)	L3 VNI	Transit VLAN	Subnet	Gateway (anycast)	Internet
TENANT1	10	10010	10000	4001	10.10.10.0/24	10.10.10.1	✓
TENANT1	20	10020	10000	4001	10.20.20.0/24	10.20.20.1	✓
TENANT2	30	10030	20000	4002	172.16.30.0/24	172.16.30.1	✗
TENANT3	50	10050	30000	4003	172.16.50.0/24	172.16.50.1	✗

5.2 EVPN Route-Targets

VRF / VLAN	Direction	Route-Target
VLAN 10 (L2)	import + export	10010:10010
VLAN 20 (L2)	import + export	10020:10020
VLAN 30 (L2)	import + export	10030:10030
VLAN 50 (L2)	import + export	10050:10050
TENANT1 (L3)	import + export	10000:10000
TENANT2 (L3)	import + export	20000:20000
TENANT3 (L3)	import + export	30000:30000

5.3 Route-Distinguishers (per VTEP per VRF)

Format: <loopback0>:<l3-vni> — e.g., leaf-01 TENANT1 = 10.0.0.11:10000

VTEP	TENANT1 RD	TENANT2 RD	TENANT3 RD
leaf-01	10.0.0.11:10000	10.0.0.11:20000	10.0.0.11:30000
leaf-02	10.0.0.12:10000	10.0.0.12:20000	10.0.0.12:30000
leaf-03	10.0.0.13:10000	10.0.0.13:20000	10.0.0.13:30000
leaf-04	10.0.0.14:10000	10.0.0.14:20000	10.0.0.14:30000
leaf-05	10.0.0.35:10000	10.0.0.35:20000	10.0.0.35:30000
leaf-06	10.0.0.36:10000	10.0.0.36:20000	10.0.0.36:30000
leaf-07	10.0.0.37:10000	10.0.0.37:20000	10.0.0.37:30000
leaf-08	10.0.0.38:10000	10.0.0.38:20000	10.0.0.38:30000
border-leaf-01	10.0.0.21:10000	10.0.0.21:20000	10.0.0.21:30000
border-leaf-02	10.0.0.22:10000	10.0.0.22:20000	10.0.0.22:30000

5.4 Vxlan1 Interface Template (same on every VTEP)

```
interface Vxlan1
  vxlan source-interface Loopback0
  vxlan udp-port 4789
  vxlan vlan 10 vni 10010    ! L2 VNI — TENANT1 VLAN10
  vxlan vlan 20 vni 10020    ! L2 VNI — TENANT1 VLAN20
  vxlan vlan 30 vni 10030    ! L2 VNI — TENANT2 VLAN30
  vxlan vlan 50 vni 10050    ! L2 VNI — TENANT3 VLAN50
  vxlan vrf TENANT1 vni 10000 ! L3 VNI — NO matching vxlan vlan line
  vxlan vrf TENANT2 vni 20000
  vxlan vrf TENANT3 vni 30000
```

Part 6 — Host PC Reference

Pod-1 Hosts (pc1 – pc16)

PC	Container	Leaf	Port	VRF	VLAN	IP	Gateway
pc1	clab-vxlan-evpn-pc1	leaf-01	eth3	TENANT1	10	10.10.10.101/24	10.10.10.1
pc2	clab-vxlan-evpn-pc2	leaf-01	eth4	TENANT1	20	10.20.20.101/24	10.20.20.1
pc3	clab-vxlan-evpn-pc3	leaf-02	eth3	TENANT1	10	10.10.10.102/24	10.10.10.1
pc4	clab-vxlan-evpn-pc4	leaf-02	eth4	TENANT1	20	10.20.20.102/24	10.20.20.1
pc5	clab-vxlan-evpn-pc5	leaf-03	eth3	TENANT1	10	10.10.10.103/24	10.10.10.1
pc6	clab-vxlan-evpn-pc6	leaf-03	eth4	TENANT1	20	10.20.20.103/24	10.20.20.1
pc7	clab-vxlan-evpn-pc7	leaf-04	eth3	TENANT1	10	10.10.10.104/24	10.10.10.1
pc8	clab-vxlan-evpn-pc8	leaf-04	eth4	TENANT1	20	10.20.20.104/24	10.20.20.1
pc9	clab-vxlan-evpn-pc9	leaf-01	eth5	TENANT2	30	172.16.30.101/24	172.16.30.1
pc10	clab-vxlan-evpn-pc10	leaf-02	eth5	TENANT2	30	172.16.30.102/24	172.16.30.1
pc11	clab-vxlan-evpn-pc11	leaf-03	eth5	TENANT2	30	172.16.30.103/24	172.16.30.1
pc12	clab-vxlan-evpn-pc12	leaf-04	eth5	TENANT2	30	172.16.30.104/24	172.16.30.1
pc13	clab-vxlan-evpn-pc13	leaf-01	eth6	TENANT3	50	172.16.50.101/24	172.16.50.1
pc14	clab-vxlan-evpn-pc14	leaf-02	eth6	TENANT3	50	172.16.50.102/24	172.16.50.1
pc15	clab-vxlan-evpn-pc15	leaf-03	eth6	TENANT3	50	172.16.50.103/24	172.16.50.1
pc16	clab-vxlan-evpn-pc16	leaf-04	eth6	TENANT3	50	172.16.50.104/24	172.16.50.1

Pod-2 Hosts (pc17 – pc32)

PC	Container	Leaf	Port	VRF	VLAN	IP	Gateway
pc17	clab-vxlan-evpn-pc17	leaf-05	eth3	TENANT1	10	10.10.10.105/24	10.10.10.1
pc18	clab-vxlan-evpn-pc18	leaf-05	eth4	TENANT1	20	10.20.20.105/24	10.20.20.1
pc19	clab-vxlan-evpn-pc19	leaf-05	eth5	TENANT2	30	172.16.30.105/24	172.16.30.1
pc20	clab-vxlan-evpn-pc20	leaf-05	eth6	TENANT3	50	172.16.50.105/24	172.16.50.1
pc21	clab-vxlan-evpn-pc21	leaf-06	eth3	TENANT1	10	10.10.10.106/24	10.10.10.1
pc22	clab-vxlan-evpn-pc22	leaf-06	eth4	TENANT1	20	10.20.20.106/24	10.20.20.1
pc23	clab-vxlan-evpn-pc23	leaf-06	eth5	TENANT2	30	172.16.30.106/24	172.16.30.1
pc24	clab-vxlan-evpn-pc24	leaf-06	eth6	TENANT3	50	172.16.50.106/24	172.16.50.1
pc25	clab-vxlan-evpn-pc25	leaf-07	eth3	TENANT1	10	10.10.10.107/24	10.10.10.1
pc26	clab-vxlan-evpn-pc26	leaf-07	eth4	TENANT1	20	10.20.20.107/24	10.20.20.1
pc27	clab-vxlan-evpn-pc27	leaf-07	eth5	TENANT2	30	172.16.30.107/24	172.16.30.1
pc28	clab-vxlan-evpn-pc28	leaf-07	eth6	TENANT3	50	172.16.50.107/24	172.16.50.1
pc29	clab-vxlan-evpn-pc29	leaf-08	eth3	TENANT1	10	10.10.10.108/24	10.10.10.1
pc30	clab-vxlan-evpn-pc30	leaf-08	eth4	TENANT1	20	10.20.20.108/24	10.20.20.1
pc31	clab-vxlan-evpn-pc31	leaf-08	eth5	TENANT2	30	172.16.30.108/24	172.16.30.1
pc32	clab-vxlan-evpn-pc32	leaf-08	eth6	TENANT3	50	172.16.50.108/24	172.16.50.1

Useful Ping Test Matrix

```
# Intra-leaf, intra-VRF (fastest path – local switch)
docker exec clab-vxlan-evpn-pc1 ping -c2 10.10.10.101 # pc1 → itself (loopback test)

# Intra-pod, intra-VRF (VXLAN bridging, same L2 VNI, same pod)
docker exec clab-vxlan-evpn-pc1 ping -c2 10.10.10.102 # pc1 → pc3 (both TENANT1/V10, leaf-01→leaf-02)

# Intra-pod, inter-VLAN, intra-VRF (VXLAN symmetric IRB – L3 VNI path)
docker exec clab-vxlan-evpn-pc1 ping -c2 10.20.20.101 # pc1 → pc2 (TENANT1 VLAN10 → VLAN20)

# Inter-pod, same VRF (5-stage full path: leaf→sp→SS→sp→leaf)
docker exec clab-vxlan-evpn-pc1 ping -c2 10.10.10.105 # pc1 → pc17 (pod-1 → pod-2)

# Cross-VRF (should FAIL – isolation test)
docker exec clab-vxlan-evpn-pc1 ping -c2 172.16.30.101 # TENANT1 → TENANT2 (expected: no reply)

# Internet (TENANT1 only – via border-leaf Type-5 EVPN)
docker exec clab-vxlan-evpn-pc1 ping -c2 1.1.1.1 # → isp-01 loopback
docker exec clab-vxlan-evpn-pc1 ping -c2 8.8.8.8 # → shared between ISP-01/ISP-02

# Internet from TENANT2 (should FAIL – not connected to internet)
docker exec clab-vxlan-evpn-pc9 ping -c2 1.1.1.1 # expected: no reply
```

Part 7 — EVPN Route Type Reference

Type	Name	What it carries	Generated by	Used for
Type-1	Ethernet Auto-Discovery	ESI info	Multi-homed VTEPs	Fast convergence, aliasing
Type-2	MAC/IP Advertisement	MAC + optional IP	Every VTEP with local hosts	L2 forwarding, ARP suppression
Type-3	Inclusive Multicast (IMET)	VTEP IP per VNI	Every VTEP	BUM flood list (replaces flood-and-learn)
Type-4	Ethernet Segment	ESI election	Multi-homed VTEPs	Designated forwarder election
Type-5	IP Prefix	IP prefix (host or internet)	VTEPs redistributing routed routes	Inter-VRF routing, internet distribution

In this lab: - Type-2 — MAC/IP bindings for all 32 PCs, distributed across all pods - Type-3 — VTEP membership per VNI (10010, 10020, 10030, 10050, 10000, 20000, 30000) - Type-5 — Internet routes from border-leaves (0.0.0.0/0, 1.1.1.1/32, 8.8.8.8/32, 2.2.2.2/32) propagated to all pod-1 and pod-2 leaves inside TENANT1 VRF

Part 8 — Show Commands by Role

On the Lab Host (Ubuntu)

```
# Lab health
containerlab inspect --all
docker ps --format '{{.Names}}\t{{.Status}}' | grep clab-vxlan-evpn-

# Count containers
docker ps --format '{{.Names}}' | grep -c clab-vxlan-evpn-

# RAM per container
docker stats --no-stream --format '{{.Name}}\t{{.MemUsage}}' | grep clab- | sort

# Check for stale deploy processes
ps aux | grep containerlab
```

Super-Spine (EVPN RR health check)

```
N=clab-vxlan-evpn-super-spine-01

# Underlay BGP — all 8 peers (4 pod-1 spines + 4 pod-2 spines) should be Estab
docker exec $N Cli -p15 -c "show ip bgp summary"

# EVPN overlay — same 8 peers should be Estab
docker exec $N Cli -p15 -c "show bgp evpn summary"

# Confirm next-hop-unchanged is set (VTEP IPs visible)
docker exec $N Cli -p15 -c "show bgp evpn detail" | grep -A2 "next hop"

# All EVPN routes — should see Type-2, Type-3, Type-5 from all pods
docker exec $N Cli -p15 -c "show bgp evpn"
```

Pod Spine (EVPN RR health check)

```
N=clab-vxlan-evpn-spine-01

# Underlay — should show: 6 leaves/BLs (pod-1) + 2 super-spines = 8 peers
docker exec $N Cli -p15 -c "show ip bgp summary"

# EVPN — same peers, AF EVPN
docker exec $N Cli -p15 -c "show bgp evpn summary"

# Verify VTEP IPs are distributed across pods
docker exec $N Cli -p15 -c "show bgp evpn route-type ip-prefix"
```


Leaf / Border-Leaf (VTEP verification)

```

N=clab-vxlan-evpn-leaf-01

# Underlay — 2 spines (Estab)
docker exec $N Cli -p15 -c "show ip bgp summary"

# EVPN overlay — 2 spines (Estab)
docker exec $N Cli -p15 -c "show bgp evpn summary"

# VTEP table — should list all remote VTEPs
docker exec $N Cli -p15 -c "show vxlan vtep"

# MAC/IP bindings (Type-2)
docker exec $N Cli -p15 -c "show vxlan address-table"

# VNI status
docker exec $N Cli -p15 -c "show vxlan vni"

# Sanity check (catches VNI conflicts)
docker exec $N Cli -p15 -c "show vxlan config-sanity"

# Tenant routing table
docker exec $N Cli -p15 -c "show ip route vrf TENANT1"
docker exec $N Cli -p15 -c "show ip route vrf TENANT2"
docker exec $N Cli -p15 -c "show ip route vrf TENANT3"

# Internet routes (Type-5) — only TENANT1
docker exec $N Cli -p15 -c "show bgp evpn route-type ip-prefix"

# ARP table per VRF
docker exec $N Cli -p15 -c "show arp vrf TENANT1"

# Catch silent config drops
docker exec $N Cli -p15 -c "show running-config interfaces Vxlan1"
docker exec $N Cli -p15 -c "show running-config section bgp"

```

Border-Leaf (Internet BGP check)

```

N=clab-vxlan-evpn-border-leaf-01

# ISP peers inside VRF TENANT1
docker exec $N Cli -p15 -c "show bgp summary vrf TENANT1"

# Internet prefixes received from ISPs
docker exec $N Cli -p15 -c "show ip bgp vrf TENANT1"

# What gets redistributed as Type-5
docker exec $N Cli -p15 -c "show bgp evpn route-type ip-prefix"

# Static routes toward ISPs
docker exec $N Cli -p15 -c "show ip route vrf TENANT1"

```

FRR-INET (Internet router)

```
docker exec -it clab-vxlan-evpn-frr-inet vtysh

# Inside vtysh:
show bgp summary
show bgp neighbors
show ip bgp                # full prefix table (~500 routes)
show ip route
```

Host PC (End-to-end reachability)

```
PC=clab-vxlan-evpn-pc1

docker exec $PC ping -c2 10.10.10.102 # intra-pod
docker exec $PC ping -c2 10.10.10.105 # inter-pod
docker exec $PC ping -c2 1.1.1.1      # internet (TENANT1 only)
docker exec $PC traceroute 10.10.10.105 # 5-stage path visible
docker exec $PC ip route show         # check default route
```

Part 9 — Batch Commands (All Nodes at Once)

```
# BGP underlay summary across ALL spine-tier nodes
for n in super-spine-01 super-spine-02 spine-01 spine-02 spine-03 spine-04; do
  echo "=== $n ===" && \
  docker exec clab-vxlan-evpn-$n Cli -p15 -c "show ip bgp summary" 2>&1 \
  | grep -E "Estab|Idle|Active|Total|Neighbor"
done

# EVPN summary across ALL spine-tier nodes
for n in super-spine-01 super-spine-02 spine-01 spine-02 spine-03 spine-04; do
  echo "=== $n ===" && \
  docker exec clab-vxlan-evpn-$n Cli -p15 -c "show bgp evpn summary" 2>&1 \
  | grep -E "Estab|Idle|Total|Neighbor"
done

# VTEP table from every leaf
for n in leaf-01 leaf-02 leaf-03 leaf-04 leaf-05 leaf-06 leaf-07 leaf-08 border-leaf-01 border-leaf-02; do
  echo "=== $n ===" && \
  docker exec clab-vxlan-evpn-$n Cli -p15 -c "show vxlan vtep" 2>&1
done

# Internet routes on all pod-1 leaves (Type-5)
for n in leaf-01 leaf-02 leaf-03 leaf-04; do
  echo "=== $n ===" && \
  docker exec clab-vxlan-evpn-$n Cli -p15 -c "show bgp evpn route-type ip-prefix" 2>&1 \
  | grep -E "0\.\0\.\0\.\0|1\.\1\.\1\.\1|8\.\8\.\8\.\8|2\.\2\.\2\.\2"
done

# Ping inter-pod from all TENANT1 pod-1 hosts
for pc in pc1 pc3 pc5 pc7; do
  echo "=== $pc ===" && docker exec clab-vxlan-evpn-$pc ping -c1 -W1 10.10.10.105 2>&1 | tail -2
done
```

Part 10 — Troubleshooting Flowchart

Step 1 — Is the lab running?

```
docker ps --format '{{.Names}}' | grep -c clab-vxlan-evpn-
# Expected: 51
```

If < 51 → run `containerlab deploy --topo /root/VXLAN-EVPN-Containerlab/topology.clab.yml`

Step 2 — Do interfaces exist?

```
docker exec clab-vxlan-evpn-spine-01 Cli -p15 -c "show interfaces status"
# Should show: Ethernet1-8, Loopback0, Management0
```

If only Management0 appears → containers were started with `docker start` (veth pairs missing). **Fix:** `containerlab destroy --cleanup && containerlab deploy --topo topology.clab.yml`

Step 3 — Underlay BGP up?

```
docker exec clab-vxlan-evpn-spine-01 Cli -p15 -c "show ip bgp summary"
# All neighbors: Estab
```

If `Idle` → check P2P link, IP config, AS numbers

If `Active` → TCP not forming, check interface status

If `OpenSent/OpenConfirm` → AS mismatch, check `remote-as`

Step 4 — EVPN overlay up?

```
docker exec clab-vxlan-evpn-spine-01 Cli -p15 -c "show bgp evpn summary"
```

If underlay is up but EVPN is not: - Check `send-community extended` on both sides - Check `update-source Loopback0` + `ebgp-multihop 3` - Check that Lo0 is reachable in underlay (`show ip bgp 10.0.0.11`)

Step 5 — VXLAN data plane?

```
docker exec clab-vxlan-evpn-leaf-01 Cli -p15 -c "show vxlan vtep"
# Should show all remote VTEPs: 10.0.0.12, .13, .14, .21, .22, .35-.38
```

Missing VTEPs → check EVPN overlay, route-targets, Type-3 routes

```
docker exec clab-vxlan-evpn-leaf-01 Cli -p15 -c "show bgp evpn route-type imet"
# One Type-3 per VTEP per L2 VNI
```

Step 6 — Host can't ping?

```
docker exec clab-vxlan-evpn-pc1 ping -c2 10.10.10.102
```

1. Check host IP + default route: `docker exec clab-vxlan-evpn-pc1 ip addr; ip route`
2. Check leaf ARP: `docker exec clab-vxlan-evpn-leaf-01 Cli -p15 -c "show arp vrf TENANT1"`
3. Check Type-2 on remote leaf: `show vxlan address-table`
4. Check VRF routing: `show ip route vrf TENANT1`

Step 7 — Internet not working from TENANT1?

```
docker exec clab-vxlan-evpn-pc1 ping -c2 1.1.1.1
```

Check in order: 1. Border-leaf ISP sessions: `show bgp summary vrf TENANT1` on border-leaf-01 2. Type-5 routes generated: `show bgp evpn route-type ip-prefix` on border-leaf-01 3. Type-5 routes received at leaf: `show bgp evpn route-type ip-prefix` on leaf-01 4. Route in VRF table: `show ip route vrf TENANT1` (look for 0.0.0.0/0) 5. PERMIT_ALL route-map exists: `show route-map PERMIT_ALL`

Common mistake: `redistribute connected` without route-map → silently ignored by EOS.

Fix: `redistribute connected route-map PERMIT_ALL`

Part 11 — cEOS Silent-Rejection Cheat Sheet

What you typed	What EOS quietly did	How to detect	Fix
<code>vxlan vrf X vni Y</code> when Y already on <code>vxlan vlan</code>	Silently drops the vrf line	<code>show vxlan config-sanity</code>	Remove the <code>vxlan vlan X vni Y</code> line
<code>redistribute connected</code> alone in VRF BGP AF	Accepted by parser, ignored at runtime	No Type-5 routes appear	Add <code>route-map PERMIT_ALL</code>
<code>address-family ipv4 unicast</code>	Rejected (unicast keyword not valid on 4.36)	Config not saved	Use <code>address-family ipv4</code>
<code>network 0.0.0.0/0</code> under VRF AF	Sometimes silently dropped	Check <code>show bgp evpn</code> for Type-5	Use <code>redistribute static route-map PERMIT_ALL</code> with a static default
<code>ebgp-multihop</code> without <code>update-source</code>	EVPN session uses physical interface IP (not Lo0)	EVPN comes up but VTEP IPs are wrong	Always pair with <code>update-source Loopback0</code>

Defensive practice after every config push:

```
show running-config interfaces Vxlan1
show running-config section bgp
show vxlan config-sanity
```

Part 12 — Containerlab Operations

```
# Deploy (or redeploy after startup) – ALWAYS use this
cd /root/VXLAN-EVPN-Containerlab
containerlab deploy --topo topology.clab.yml

# Destroy + wipe state (before a fresh rebuild)
containerlab destroy --topo topology.clab.yml --cleanup

# Check what's running
containerlab inspect --all

# NEVER do this (veth pairs are lost):
docker restart clab-vxlan-evpn-spine-01 # ✗
docker start clab-vxlan-evpn-spine-01 # ✗

# Save config from inside EOS before destroying
docker exec clab-vxlan-evpn-leaf-01 Cli -p15 -c "write memory"
docker exec clab-vxlan-evpn-leaf-01 cat /mnt/flash/startup-config \
> /root/VXLAN-EVPN-Containerlab/configs/LEAF1.cfg
```

Portal Operations

```
# Start portal
kill -f 'python.*app.py' 2>/dev/null
cd /root/lab-portal
nohup venv/bin/python app.py > /tmp/portal.log 2>&1 &

# Check portal is up
curl -s -o /dev/null -w "%{http_code}" http://localhost:5000

# Live log
tail -f /tmp/portal.log
```

Part 13 — Data Plane Walk-Through: pc1 → pc17

End-to-end path for a packet from pc1 (pod-1, leaf-01) to pc17 (pod-2, leaf-05). Both are in TENANT1 / VLAN10.

```

pc1 (10.10.10.101)
| eth1 → leaf-01 eth3 (access VLAN10)
▼
leaf-01
1. Looks up dst 10.10.10.105 in VRF TENANT1
2. Finds EVPN Type-2: 10.10.10.105 → MAC of pc17 → VTEP 10.0.0.35 (leaf-05)
3. ARP suppression: answers locally from EVPN binding (no flood needed)
4. Encapsulates: outer src=10.0.0.11, dst=10.0.0.35, UDP/4789, VNI=10010
|
| VXLAN frame → underlay routing
▼
spine-01 or spine-02 (ECMP hash selects one)
| routes 10.0.0.35 → next-hop super-spine (via Lo0 BGP advertisement)
▼
super-spine-01 or super-spine-02 (next-hop-unchanged preserves 10.0.0.35)
| routes 10.0.0.35 → spine-03 or spine-04
▼
spine-03 or spine-04
| routes 10.0.0.35 → leaf-05
▼
leaf-05
5. Decapsulates VXLAN (VNI 10010 → VLAN10 → TENANT1)
6. Delivers to pc17 on eth3
▼
pc17 (10.10.10.105)

EVPN control-plane AS path for this route at leaf-01:
65000 → 65500 → 65010 → 65031
(pod-1 spine → super-spine → pod-2 spine → leaf-05)

Data plane hops: 5 (leaf-01 → sp → SS → sp → leaf-05)
Control plane hops: 4 AS hops

```

Symmetric IRB: pc1 → pc2 (inter-VLAN, same VRF)

```

pc1 (10.10.10.101, VLAN10) wants to reach pc2 (10.20.20.101, VLAN20)

leaf-01 (local):
- Receives frame on VLAN10
- Routing lookup in TENANT1 VRF: dst 10.20.20.101
- EVPN Type-2: MAC of pc2 → VTEP 10.0.0.11 (itself! pc2 is on same leaf)
- Routes locally via SVI VLAN20 → delivers to pc2

If pc2 were on leaf-02:
- leaf-01 encaps with L3 VNI 10000 (not L2 VNI 10010/10020)
- Inner frame: routed from VLAN10 SVI to VLAN20
- Outer VXLAN: VNI=10000, dst=leaf-02 Lo0
- leaf-02 decaps, routes in TENANT1, delivers on VLAN20

```

Part 14 — Config Templates by Role

Super-Spine Template

```

service routing protocols model multi-agent
!
router bgp 65500
  router-id <Lo0>
  no bgp default ipv4-unicast
  maximum-paths 4 ecmp 4
  neighbor SPINE_EVPN peer group
  neighbor SPINE_EVPN update-source Loopback0
  neighbor SPINE_EVPN ebgp-multihop 3
  neighbor SPINE_EVPN send-community extended
  neighbor SPINE_EVPN next-hop-unchanged      ! <-- CRITICAL for VTEP preservation
  neighbor SPINE_EVPN maximum-routes 0
  neighbor SPINE_UNDERLAY peer group
  neighbor SPINE_UNDERLAY send-community extended
  neighbor SPINE_UNDERLAY maximum-routes 12000
  ! ... neighbor statements ...
  address-family evpn
    neighbor SPINE_EVPN activate
  address-family ipv4
    neighbor SPINE_UNDERLAY activate
    network <Lo0>/32

```

Pod-Spine Template

```

router bgp <pod-ASN>
  router-id <Lo0>
  no bgp default ipv4-unicast
  maximum-paths 4 ecmp 4
  neighbor LEAF_EVPN peer group
  neighbor LEAF_EVPN update-source Loopback0
  neighbor LEAF_EVPN ebgp-multihop 3
  neighbor LEAF_EVPN send-community extended
  neighbor LEAF_EVPN maximum-routes 0
  ! (no next-hop-unchanged here – spines are not the top)
  neighbor LEAF_UNDERLAY peer group
  neighbor LEAF_UNDERLAY send-community extended
  neighbor LEAF_UNDERLAY maximum-routes 12000
  neighbor SS_EVPN peer group
  neighbor SS_EVPN update-source Loopback0
  neighbor SS_EVPN ebgp-multihop 3
  neighbor SS_EVPN send-community extended
  neighbor SS_EVPN next-hop-unchanged      ! <-- pass VTEP IPs up to SS unchanged
  neighbor SS_EVPN maximum-routes 0
  ! ... neighbor statements ...
  address-family evpn
    neighbor LEAF_EVPN activate
    neighbor SS_EVPN activate
  address-family ipv4
    neighbor LEAF_UNDERLAY activate
    neighbor SS_UNDERLAY activate
    network <Lo0>/32

```

Leaf VTEP Template

```

vlan 10,20,30,50,4001,4002,4003
vrf instance TENANT1
vrf instance TENANT2
vrf instance TENANT3
!
interface Vxlan1
    vxlan source-interface Loopback0
    vxlan udp-port 4789
    vxlan vlan 10 vni 10010
    vxlan vlan 20 vni 10020
    vxlan vlan 30 vni 10030
    vxlan vlan 50 vni 10050
    vxlan vrf TENANT1 vni 10000    ! No vxlan vlan line for VNI 10000
    vxlan vrf TENANT2 vni 20000
    vxlan vrf TENANT3 vni 30000
!
ip virtual-router mac-address 00:1c:73:00:00:99
!
router bgp <leaf-ASN>
    vlan 10
        rd auto
        route-target both 10010:10010
        redistribute learned
    vlan 20
        rd auto
        route-target both 10020:10020
        redistribute learned
    ! ... vlan 30, 50 ...
    address-family evpn
        neighbor SPINE_EVPN activate
    vrf TENANT1
        rd <Lo0>:10000
        route-target import evpn 10000:10000
        route-target export evpn 10000:10000
        redistribute connected route-map PERMIT_ALL    ! route-map REQUIRED
    ! ... TENANT2, TENANT3 ...

```

Border-Leaf Addition (on top of Leaf template)

```

interface Ethernet3
    description >>> ISP-01 uplink <<<
    no switchport
    vrf TENANT1
    ip address 100.64.0.2/30

route-map PERMIT_ALL permit 10
!
router bgp 65021
    vrf TENANT1
        neighbor 100.64.0.1 remote-as 65100
        address-family ipv4
            neighbor 100.64.0.1 activate
            redistribute connected route-map PERMIT_ALL
            redistribute static    route-map PERMIT_ALL

```


Appendix A — File Locations

Item	Path
Topology file	<code>/root/VXLAN-EVPN-Containerlab/topology.clab.yml</code>
cEOS configs	<code>/root/VXLAN-EVPN-Containerlab/configs/</code>
FRR config	<code>/root/VXLAN-EVPN-Containerlab/configs/frr-inet/frr.conf</code>
Portal app	<code>/root/lab-portal/app.py</code>
Portal venv	<code>/root/lab-portal/venv/</code>
Portal templates	<code>/root/lab-portal/templates/</code>
Packet captures	<code>/root/lab-portal/captures/</code>
Portal log	<code>/tmp/portal.log</code>
GitHub repo	https://github.com/Harinischal/VXLAN-EVPN-Lab

Appendix B — Number Summary

Loopback IPs:

10.0.0.1	spine-01	10.0.0.2	spine-02
10.0.0.3	spine-03	10.0.0.4	spine-04
10.0.0.11	leaf-01	10.0.0.12	leaf-02
10.0.0.13	leaf-03	10.0.0.14	leaf-04
10.0.0.21	border-leaf-01	10.0.0.22	border-leaf-02
10.0.0.35	leaf-05	10.0.0.36	leaf-06
10.0.0.37	leaf-07	10.0.0.38	leaf-08
10.0.0.91	super-spine-01	10.0.0.92	super-spine-02

Underlay subnets:

10.1.1.x/31	spine-01 ↔ pod-1 leaves / border-leaves
10.1.2.x/31	spine-02 ↔ pod-1 leaves / border-leaves
10.1.3.x/31	spine-03 ↔ pod-2 leaves
10.1.4.x/31	spine-04 ↔ pod-2 leaves
10.2.1.x/31	super-spine ↔ pod-1 spines (SS-01 side)
10.2.2.x/31	super-spine ↔ pod-1 spines (SS-02 side)
10.2.3.x/31	super-spine ↔ pod-2 spines (SS-01 side)
10.2.4.x/31	super-spine ↔ pod-2 spines (SS-02 side)
100.64.0.x/30	BL-01/02 ↔ ISP-01
100.64.1.x/30	BL-01/02 ↔ ISP-02
10.99.0.x/30	ISP-01/02 ↔ frr-inet

VNIs:

10010	VLAN10 L2 (TENANT1)
10020	VLAN20 L2 (TENANT1)
10030	VLAN30 L2 (TENANT2)
10050	VLAN50 L2 (TENANT3)
10000	TENANT1 L3 VNI
20000	TENANT2 L3 VNI
30000	TENANT3 L3 VNI

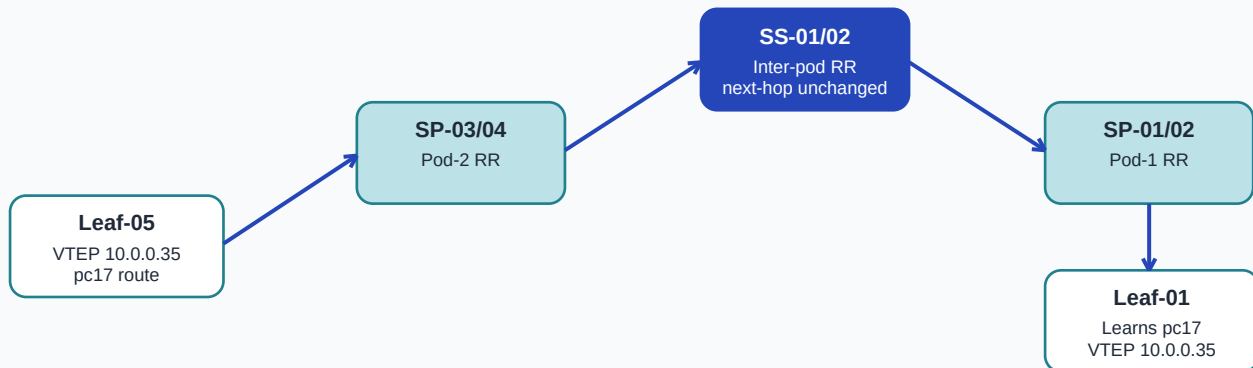
Transit VLANs (L3 VNI handoff):

4001 → TENANT1	4002 → TENANT2	4003 → TENANT3
----------------	----------------	----------------

Visual Appendix — Control Plane vs Data Plane

Same topology, two different jobs: EVPN control-plane advertisement and VXLAN data-plane forwarding.

EVPN control plane: routes are reflected through spines and super-spines



Type-2 MAC/IP, Type-3 IMET, Type-5 prefixes

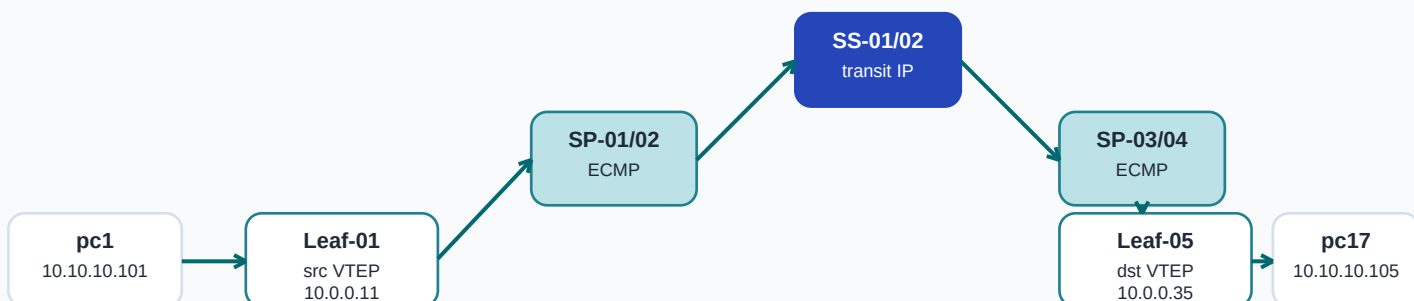
Why next-hop-unchanged matters

Super-spines preserve the original VTEP loopback in reflected EVPN routes. Leaf-01 still sees leaf-05 (10.0.0.35) as the VXLAN destination, not the super-spine.

Expected BGP view at leaf-01

The EVPN control-plane AS path toward pc17 is 65000 → 65500 → 65010 → 65031. The next-hop remains the remote VTEP loopback.

VXLAN data plane: packet follows underlay ECMP directly toward the remote VTEP



Outer VXLAN: src 10.0.0.11 → dst 10.0.0.35, UDP/4789, VNI 10010

Troubleshooting order

1. Underlay reaches all Loopback0s.
2. EVPN neighbors are established.
3. Type-3 IMET creates VTEP membership.
4. Type-2 MAC/IP maps host to VTEP.

Key separation

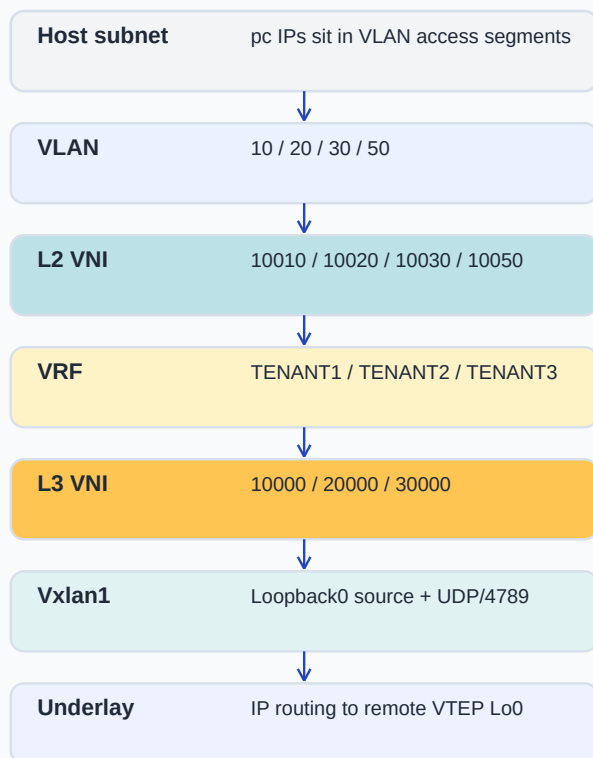
EVPN route-reflectors carry control-plane routes only. The actual VXLAN-encapsulated frame is routed by the IP underlay and is not "switched" by the EVPN RR function.

Visual Appendix — Tenant, VLAN, VNI, and Gateway Map

How VLANs become L2 VNIs, how VRFs become L3 VNIs, and which tenant has internet reachability.

Tenant	VLAN(s)	L2 VNI(s)	L3 VNI	Anycast gateway	Internet
TENANT1	10, 20	10010, 10020	10000	10.10.10.1 / 10.20.20.1	Yes
TENANT2	30	10030	20000	172.16.30.1	No
TENANT3	50	10050	30000	172.16.50.1	No

Per-VTEP service stack



Same VLAN, same tenant

Uses the relevant L2 VNI. Example: pc1 VLAN10 to pc17 VLAN10 uses VNI 10010 and Type-2/Type-3 EVPN information.

Different VLAN, same tenant

Uses symmetric IRB with the tenant L3 VNI. Example: TENANT1 VLAN10 to VLAN20 uses L3 VNI 10000 between VTEPs.

Different tenant

Blocked unless explicit route leaking is configured. TENANT1, TENANT2, and TENANT3 are separate VRFs in this lab.

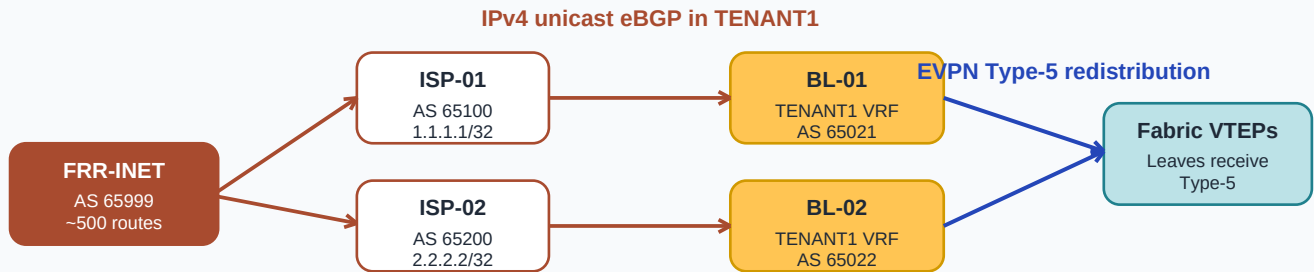
Internet

Only TENANT1 learns Type-5 internet/default prefixes from the border-leaves. TENANT2 and TENANT3 should fail internet tests.

Practical check: if VXLAN service behavior is wrong, compare **show vxlan vni**, **show vxlan address-table**, route-targets, and the Vxlan1 template. Do not reuse an L3 VNI as a VLAN VNI; EOS can silently drop the VRF line.

Visual Appendix — TENANT1 Internet and Type-5 Prefix Flow

How ISP routes enter TENANT1 and become EVPN Type-5 routes for all VTEPs.



What should appear where

On border-leaf

```
show bgp summary vrf TENANT1
ISP peers Established.
```

```
show ip bgp vrf TENANT1
```

EVPN export

```
show bgp evpn route-type ip-prefix
Default and ISP loopbacks appear as Type-5.
```

On leaf

```
show ip route vrf TENANT1
0.0.0.0/0 or selected internet
prefixes are installed.
```

Expected ping behavior

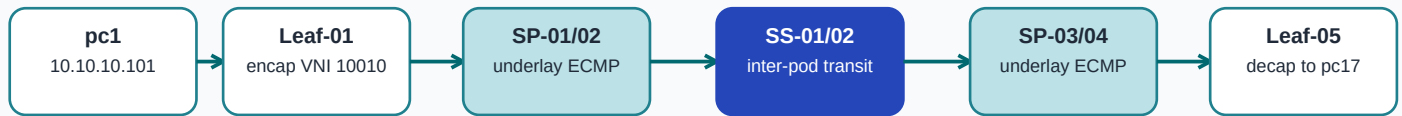
Source	Destination	Expected	Why
pc1 TENANT1	1.1.1.1	PASS	TENANT1 receives Type-5 internet routes
pc1 TENANT1	8.8.8.8	PASS	Advertised by ISP-01/ISP-02
pc9 TENANT2	1.1.1.1	FAIL	TENANT2 has no internet propagation
pc13 TENANT3	1.1.1.1	FAIL	TENANT3 has no internet propagation

Common failure point

If redistribute connected or redistribute static is configured without route-map PERMIT_ALL, the parser may accept the syntax but Type-5 routes may not appear. Confirm with show bgp evpn route-type ip-prefix and show route-map PERMIT_ALL.

Visual Appendix — pc1 → pc17 Packet Walk-Through

End-to-end packet intuition for TENANT1 VLAN10 across Pod-1 to Pod-2.



Outer tunnel: 10.0.0.11 → 10.0.0.35 | UDP 4789 | VNI 10010

Step	Device	What happens	Validate
1	leaf-01	Receives VLAN10 frame from pc1 and looks up destination in TENANT1.	show ip route vrf TENANT1
2	leaf-01	Finds EVPN Type-2 binding: pc17 is behind VTEP 10.0.0.35.	show vxlan address-table
3	leaf-01	Encapsulates into VXLAN: src 10.0.0.11, dst 10.0.0.35.	show vxlan vtep
4	underlay	Spines and super-spines route the outer IP packet using ECMP.	traceroute 10.10.10.105
5	leaf-05	Decapsulates VNI 10010 into VLAN10/TENANT1 and forwards to pc17.	show vxlan vni

Control plane versus packet path

The EVPN AS path may show multiple control-plane hops, but the packet is simply IP-routed toward the destination VTEP loopback. RRs advertise reachability; they do not become VXLAN endpoints.

ARP suppression note

For known remote hosts, leaf-01 can answer ARP using EVPN Type-2 MAC/IP bindings. This avoids flooding and keeps the fabric deterministic.

Mini decision tree for failed pc1 → pc17

If traceroute stops before the remote pod: check underlay BGP and Lo0 routes.

If underlay is good but VTEP missing: check EVPN neighbors, Type-3 IMET, and route-targets.

If VTEP exists but host fails: check Type-2 MAC/IP, VLAN/VNI mapping, SVI gateway, and host default route.